Finance Cost Reporting - Snowflake

Business Context - Problem and Solution:

Finance controllers receive thousands of accounting line items synced daily from SAP. To report costs by management category (e.g. Wages & Salaries, EU Funded Projects, Maintenance, etc.), they map the document by hand in Excel, using their own rules. This is slow and inconsistent, and it's hard to verify the output. Data problems such as duplicate postings or cost centers missing from organization hierarchy surface only at month-end close.

Solution - Snowflake native pipeline that:
- Starts from the SAP documents in Snowflake (for this exercise I have generated synthetic data)
- Turns them into a cleansed, enriched reporting dataset that refreshes daily, with validation before anything is published
- Classifies every line item into a cost type using rules that controllers maintain themselves, uploaded from Excel, so a rule change needs no code change
- Presents the result in a Streamlit app with filters, drill-down to single documents, a data-quality work list and an admin page for the rules.

Results & value added:
- Every cost line is either classified or explicitly marked Unclassified, so they equals the total - the finance controllers can check the results against local SAPs reports
- Every classification can be traced to a rule
- Data quality issues are visible daily, not at month close.

Possible extensions:
- AI Layer: talk to your data using Cortex - finance users can ask questions in plain language
  - Having the base structure of the project well defined, the agents can be easily implemented in the solution: build the semantic layer on top of the data, create the agent using the tool and evaluate the agent performance against a predefined test data.
- Real SAP ingestion - can be fetched from SAP daily using some custom jobs / SAP connectors
- Operational alerting: send email when tasks fail or when data-quality errors rise.

Why Snowflake:
- One platform - data, transformations, scheduling, UI
- No data movement, everything runs inside Snowflake
- Role - based access management
- AI - ready
- Easy to maintain, easy to scale
- Cost effective - pay for what runs

Implementation

Create database, schemas and raw tables

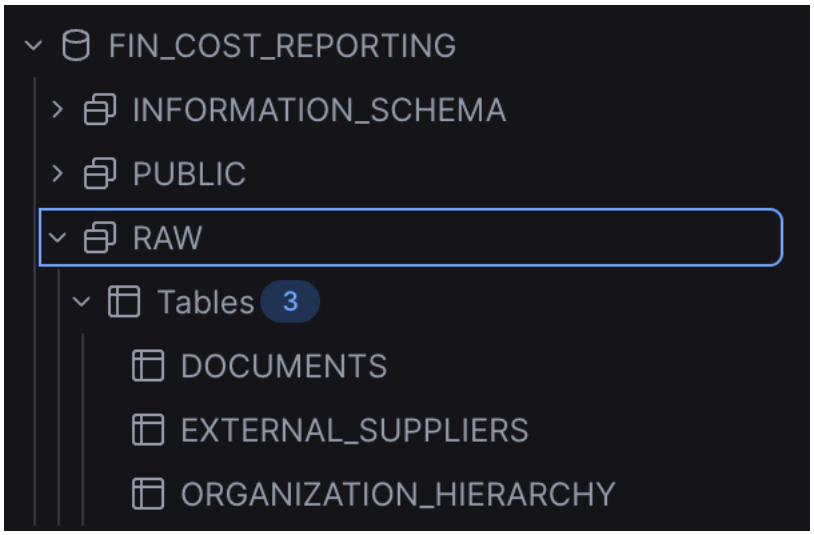
```
CREATE DATABASE IF NOT EXISTS FIN_COST_REPORTING;
CREATE SCHEMA   IF NOT EXISTS FIN_COST_REPORTING.RAW;

USE SCHEMA FIN_COST_REPORTING.RAW;

CREATE OR REPLACE TABLE ORGANIZATION_HIERARCHY (
    PROFIT_CENTER  VARCHAR(10)  NOT NULL,
    COST_CENTER    VARCHAR(10)  NOT NULL,
    ORG_UNIT       VARCHAR(10)  NOT NULL,
    NAME           VARCHAR(100) NOT NULL,
    PRIMARY KEY (COST_CENTER)
);

CREATE OR REPLACE TABLE EXTERNAL_SUPPLIERS (
    ID    VARCHAR(10)  NOT NULL,
    NAME  VARCHAR(100) NOT NULL,
    PRIMARY KEY (ID)
);

CREATE OR REPLACE TABLE DOCUMENTS (
    ID                    VARCHAR(10)  NOT NULL,
    FISCAL_YEAR           NUMBER(4,0)  NOT NULL,
    BOOKING_PERIOD        NUMBER(2,0)  NOT NULL,
    ACCOUNT_NUMBER        VARCHAR(10)  NOT NULL,
    PROFIT_CENTER         VARCHAR(10),
    COST_CENTER           VARCHAR(10),
    FUNDING_SOURCE        VARCHAR(10),
    AMOUNT                NUMBER(18,2) NOT NULL,
    EXTERNAL_SUPPLIER_ID  VARCHAR(10),
    DOCUMENT_DETAILS      VARCHAR(200),
    PRIMARY KEY (ID)
);
```



Add synthetic data

Organization Hierarchy Data
Maps each cost center to its profit center and organizational unit. Used to enrich documents and to check that no document uses an unknown center.

```
INSERT INTO ORGANIZATION_HIERARCHY (PROFIT_CENTER, COST_CENTER, ORG_UNIT, NAME) VALUES
    ('PC1000', 'CC1001', 'HYD', 'HYDRO PLANTS – OPERATIONS'),
……..
```

External Suppliers Data
The list of external suppliers. Tells us whether a document comes from an external party and gives the supplier name.

```
INSERT INTO EXTERNAL_SUPPLIERS (ID, NAME) VALUES
    ('S001', 'CARPATHIA POWER EXCHANGE'),
……..
```

Documents data
The cost documents (postings) synced from SAP: period, account, profit and cost center, funding source, amount, supplier and description.

- Create 1200 test documents, SAP-like data
- 12 kinds of cost - each contains an account number, a description in English, German and Romanian and amount range
- 1200 numbered rows, shared out evenly over the 12 kind of costs
- Each document gets:
  - A posting month between January 2025 and October 2026
  - One of 12 cost centers, with the profit center taken from organization hierarchy
  - Funding Source
  - Amount within the range of its cost kind
  - External supplier (or none for internal docs)
  - A description in EN, DE or RO
- Data is the same each time, so the results of the checks and exercises can be compared and repeated

```
    INSERT INTO DOCUMENTS
    (ID, FISCAL_YEAR, BOOKING_PERIOD, ACCOUNT_NUMBER, PROFIT_CENTER, COST_CENTER, FUNDING_SOURCE,
     AMOUNT, EXTERNAL_SUPPLIER_ID, DOCUMENT_DETAILS)
WITH cost_kinds AS (
    SELECT * FROM (VALUES
        ( 0, '600000', 'Electricity purchase',  'Stromeinkauf',                'Achiziție energie electrică',  TRUE,  20000,  400000),
        ( 1, '610000', 'Natural gas purchase',  'Erdgaseinkauf',               'Achiziție gaze naturale',      TRUE,  10000,  250000),
        ( 2, '630000', 'Equipment maintenance', 'Wartung von Anlagen',         'Mentenanță echipamente',       TRUE,    800,   60000),
        ( 3, '640000', 'Software licences',     'Softwarelizenzen',            'Licențe software',             TRUE,    300,   40000),
        ( 4, '641000', 'IT support services',   'IT-Supportleistungen',        'Servicii de suport IT',        TRUE,    500,   25000),
        ( 5, '650000', 'Consulting services',   'Beratungsleistungen',         'Servicii de consultanță',      TRUE,   1500,   90000),
        ( 6, '660000', 'Office rent',           'Büromiete',                   'Chirie birouri',               TRUE,   2000,   20000),
        ( 7, '670000', 'Business travel',       'Dienstreisen',                'Deplasări de serviciu',        TRUE,     40,    2500),
        ( 8, '681000', 'Salaries',              'Löhne und Gehälter',          'Salarii',                      FALSE,  4000,  140000),
        ( 9, '682000', 'Social contributions',  'Sozialversicherungsbeiträge', 'Contribuții sociale',          FALSE,   900,   30000),
        (10, '684000', 'Training',              'Schulungen',                  'Training',                     TRUE,    250,   12000),
        (11, '690000', 'Depreciation',          'Abschreibungen',              'Amortizare',                   FALSE,  1500,   85000)
    ) AS k (KIND_ID, ACCOUNT_NUMBER, TEXT_EN, TEXT_DE, TEXT_RO, HAS_SUPPLIER, MIN_AMOUNT, MAX_AMOUNT)
),
postings AS (
    SELECT
        n.N,
        k.*,
        DATEADD(MONTH, MOD(ABS(HASH(n.N, 'month')), 22), DATE '2025-01-01') AS POSTING_MONTH   -- Jan 2025 ... Oct 2026
    FROM (SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) AS N FROM TABLE(GENERATOR(ROWCOUNT => 1200))) n
    JOIN cost_kinds k
      ON k.KIND_ID = MOD(n.N, 12)
)
SELECT
    'D' || LPAD(p.N::VARCHAR, 6, '0')                                   AS ID,
    YEAR(p.POSTING_MONTH)                                               AS FISCAL_YEAR,
    MONTH(p.POSTING_MONTH)                                              AS BOOKING_PERIOD,
    p.ACCOUNT_NUMBER,
    h.PROFIT_CENTER,
    h.COST_CENTER,
    CASE
        WHEN MOD(ABS(HASH(p.N, 'funding')), 10) < 6 THEN 'OWN'
        WHEN MOD(ABS(HASH(p.N, 'funding')), 10) < 8 THEN 'EU'
        ELSE 'NATIONAL'
    END                                                                 AS FUNDING_SOURCE,
    ROUND(p.MIN_AMOUNT + (p.MAX_AMOUNT - p.MIN_AMOUNT) * MOD(ABS(HASH(p.N, 'amount')), 1000) / 1000, 2)
                                                                        AS AMOUNT,
    IFF(p.HAS_SUPPLIER, 'S' || LPAD((1 + MOD(ABS(HASH(p.N, 'supplier')), 15))::VARCHAR, 3, '0'), NULL)
                                                                        AS EXTERNAL_SUPPLIER_ID,
    CASE MOD(ABS(HASH(p.N, 'language')), 3)
        WHEN 0 THEN p.TEXT_EN
        WHEN 1 THEN p.TEXT_DE
        ELSE p.TEXT_RO
    END                                                                 AS DOCUMENT_DETAILS
FROM postings p
JOIN ORGANIZATION_HIERARCHY h
  ON h.COST_CENTER = 'CC' || (1001 + MOD(ABS(HASH(p.N, 'cost_center')), 12))::VARCHAR;
```



	ID	FISCAL_YEAR	BOOKING_PERIOD	ACCOUNT_NUMBER	PROFIT_CENTER	COST_CENTER	FUNDING_SOURCE	AMOUNT	EXTERNAL_SUPPLIER_ID	DOCUMENT_DETAILS
	D000002 0.1% D000003 0.1% +98 more	2025 2026	1 12	600000 8.3% 610000 8.3% +10 more	PC9000 41.1% PC1000 23.4% +2 more	CC1007 9.8% CC1006 9.3% +10 more	OWN 62.7% NATIONAL 20.2% +1 more	153.16 391640	S012 5.9% S011 5.8% +13 more	Training 5.6% Servicii de suport IT 3.6% +33 more
1	D000001	2025	10	610000	PC1000	CC1003	NATIONAL	79840.00	S011	Achiziție gaze naturale
2	D000002	2026	2	630000	PC9000	CC1008	OWN	24420.80	S014	Equipment maintenance
3	D000003	2025	12	640000	PC1000	CC1002	OWN	37975.30	S005	Softwarelizenzen
4	D000004	2025	2	641000	PC1000	CC1002	OWN	7262.00	S005	IT-Supportleistungen

Exercise 1 - CREATE VIEW

```
CREATE SCHEMA IF NOT EXISTS CORE;
```

```
CREATE OR REPLACE VIEW CORE.V_DOCUMENTS_ENRICHED AS
WITH supplier_spend AS (
    SELECT
        EXTERNAL_SUPPLIER_ID,
        FISCAL_YEAR,
        COUNT(*)    AS SUPPLIER_DOCUMENTS_IN_YEAR,
        SUM(AMOUNT) AS SUPPLIER_SPEND_IN_YEAR
    FROM RAW.DOCUMENTS
    WHERE EXTERNAL_SUPPLIER_ID IS NOT NULL
    GROUP BY EXTERNAL_SUPPLIER_ID, FISCAL_YEAR
),
documents AS (
    SELECT
        d.*,
        DATE_FROM_PARTS(d.FISCAL_YEAR, d.BOOKING_PERIOD, 1) AS PERIOD_START_DATE
    FROM RAW.DOCUMENTS d
    WHERE d.FISCAL_YEAR >= (SELECT MAX(FISCAL_YEAR) - 1 FROM RAW.DOCUMENTS)
      AND d.BOOKING_PERIOD BETWEEN 1 AND 12
)
SELECT
    d.ID                                                    AS DOCUMENT_ID,

    -- Period
    d.FISCAL_YEAR,
    d.BOOKING_PERIOD,
    d.FISCAL_YEAR || '-' || LPAD(d.BOOKING_PERIOD::VARCHAR, 2, '0') AS PERIOD_LABEL,
    d.PERIOD_START_DATE,
    LAST_DAY(d.PERIOD_START_DATE)                           AS PERIOD_END_DATE,
    DATE_TRUNC('QUARTER', d.PERIOD_START_DATE)              AS QUARTER_START_DATE,
    DATEDIFF('month', d.PERIOD_START_DATE,
             MAX(d.PERIOD_START_DATE) OVER ())              AS MONTHS_BEFORE_LATEST_PERIOD,

    -- Account and organisation
    d.ACCOUNT_NUMBER,
    LEFT(d.ACCOUNT_NUMBER, 2)                               AS ACCOUNT_GROUP,
    d.PROFIT_CENTER,
    d.COST_CENTER,
    COALESCE(h.NAME, 'Unknown cost center')                 AS COST_CENTER_NAME,
    COALESCE(h.ORG_UNIT, 'UNMAPPED')                        AS ORG_UNIT,
    COALESCE(UPPER(TRIM(d.FUNDING_SOURCE)), 'OWN')          AS FUNDING_SOURCE,

    -- Supplier
    d.EXTERNAL_SUPPLIER_ID,
    d.EXTERNAL_SUPPLIER_ID IS NOT NULL                      AS IS_EXTERNAL_SUPPLIER,
    COALESCE(s.NAME, 'Internal posting')                    AS SUPPLIER_NAME,
    ss.SUPPLIER_DOCUMENTS_IN_YEAR,
    ss.SUPPLIER_SPEND_IN_YEAR,


    -- Amount
    d.AMOUNT,
    CASE
        WHEN d.AMOUNT >= 100000 THEN 'High (100k and more)'
        WHEN d.AMOUNT >= 10000  THEN 'Medium (10k to 100k)'
        ELSE 'Low (under 10k)'
    END                                                     AS AMOUNT_BAND,
    ROUND(100 * d.AMOUNT
          / SUM(d.AMOUNT) OVER (PARTITION BY d.COST_CENTER, d.FISCAL_YEAR), 2)
                                                            AS PCT_OF_COST_CENTER_YEAR,

    d.DOCUMENT_DETAILS
FROM documents d
LEFT JOIN RAW.ORGANIZATION_HIERARCHY h
       ON h.COST_CENTER = d.COST_CENTER
LEFT JOIN RAW.EXTERNAL_SUPPLIERS s
       ON s.ID = d.EXTERNAL_SUPPLIER_ID
LEFT JOIN supplier_spend ss
       ON ss.EXTERNAL_SUPPLIER_ID = d.EXTERNAL_SUPPLIER_ID
      AND ss.FISCAL_YEAR          = d.FISCAL_YEAR;
```



	DOCUMENT_ID	FISCAL_YEAR	BOOKING_PERIOD	PERIOD_LABEL	PERIOD_START_DATE	PERIOD_END_DATE	QUARTER_START_DATE	MONTHS_BEFORE_LATEST_PERIOD	ACCOUNT_NUMBER	ACCOUNT_GROUP
	D000192 10.0% D000240 10.0% +8 more	2025 2026	1 12	2025-05 20.0% 2025-01 10.0% +7 more	01/01/2025 01/10/2026	31/01/2025 31/10/2026	01/01/2025 01/10/2026	0 21	600000 100.0%	60 100.0%
1	D000792	2026	6	2026-06	2026-06-01	2026-06-30	2026-04-01	4	600000	60
2	D000240	2026	2	2026-02	2026-02-01	2026-02-28	2026-01-01	8	600000	60
3	D000456	2026	1	2026-01	2026-01-01	2026-01-31	2026-01-01	9	600000	60
4	D001128	2026	10	2026-10	2026-10-01	2026-10-31	2026-10-01	0	600000	60
5	D001140	2025	5	2025-05	2025-05-01	2025-05-31	2025-04-01	17	600000	60
6	D000192	2025	12	2025-12	2025-12-01	2025-12-31	2025-10-01	10	600000	60
7	D001068	2025	2	2025-02	2025-02-01	2025-02-28	2025-01-01	20	600000	60
8	D001104	2026	8	2026-08	2026-08-01	2026-08-31	2026-07-01	2	600000	60

Sources:
- RAW.DOCUMENTS
- RAW.ORGANIZATION_HIERARCHY
- RAW.EXTERNAL_SUPPLIERS

Left join between them - a document with no match is still kept in the view

supplier_spend
- adds up what each supplier was paid per fiscal year (docs number & amount)

documents
- selects what the report will contain:
  - Only last 2 fiscal years
  - Only valid booking periods (1-12)
  - Calculates the first day of each booking period

Period group:
- PERIOD_LABEL (2025-03),
- PERIOD_START_DATE,
- PERIOD_END_DATE,
- QUARTER_START_DATE
- MONTHS_BEFORE_LATEST_PERIOD (how many months before the newest period in the data)

Account and organization group:
- ACCOUNT_GROUP - first 2 digits of the account
- COST_CENTER - Unknown if not in hierarchy
- ORG_UNIT - UNMAPPED if not found in hierarchy
- FUNDING_SOURCE - trimmed, uppercase, if empty: OWN

Supplier group:
- IS_EXTERNAL_SUPPLIER (yes/no)
- SUPPLIER_NAME - if no -> Internal posting
- SUPPLIER_DOCUMENTS_IN_YEAR
- SUPPLIER_SPEND_IN_YEAR

Amount group:
- AMOUNT_BAND (high, medium, low)
- PCT_OF_COST_CENTER_YEAR - the document’s share of total of its cost center in the FY

Exercise 2 - CREATE TABLE FOR THE VIEW

CORE.DOCUMENTS_ENRICHED stores the output of the view, next exercises will read a table instead of recomputing the view on every query.

Structure:
- Same columns, same order (in addition LOADED_AT)
- Explicit types: exact decimals, sizes that match source columns, boolean for flags

```
CREATE OR REPLACE TABLE CORE.DOCUMENTS_ENRICHED (
    DOCUMENT_ID                  VARCHAR(10)   NOT NULL,

    -- PERIOD
    FISCAL_YEAR                  NUMBER(4,0)   NOT NULL,
    BOOKING_PERIOD               NUMBER(2,0)   NOT NULL,
    PERIOD_LABEL                 VARCHAR(7),
    PERIOD_START_DATE            DATE,
    PERIOD_END_DATE              DATE,
    QUARTER_START_DATE           DATE,
    MONTHS_BEFORE_LATEST_PERIOD  NUMBER(4,0),

    -- ACCOUNT AND ORGANISATION
    ACCOUNT_NUMBER               VARCHAR(10)   NOT NULL,
    ACCOUNT_GROUP                VARCHAR(2),
    PROFIT_CENTER                VARCHAR(10),
    COST_CENTER                  VARCHAR(10),
    COST_CENTER_NAME             VARCHAR(100),
    ORG_UNIT                     VARCHAR(10),
    FUNDING_SOURCE               VARCHAR(10),

    -- SUPPLIER
    EXTERNAL_SUPPLIER_ID         VARCHAR(10),
    IS_EXTERNAL_SUPPLIER         BOOLEAN,
    SUPPLIER_NAME                VARCHAR(100),
    SUPPLIER_DOCUMENTS_IN_YEAR   NUMBER(10,0),
    SUPPLIER_SPEND_IN_YEAR       NUMBER(18,2),

    -- AMOUNT
    AMOUNT                       NUMBER(18,2)  NOT NULL,
    AMOUNT_BAND                  VARCHAR(30),
    PCT_OF_COST_CENTER_YEAR      NUMBER(10,2),

    DOCUMENT_DETAILS             VARCHAR(200),


    LOADED_AT                    TIMESTAMP_NTZ NOT NULL,

    PRIMARY KEY (DOCUMENT_ID)
);
```

Exercise 3 - CREATE PROCEDURE

CORE.SP_REFRESH_DOCUMENTS_ENRICHED() replaces the content of the above created table with the current output of the view.

Steps:
- Counts the rows that are in the table
- In one transaction:
  - Deletes all rows
  - Inserts latest data
  - Checks if something was loaded
- Returns a status

If there is an error, the transaction is rolled back, so the table keeps its previous content.

Parameter P_SIMULATE_ERROR exists only to demonstrate the rollback.

```
CREATE OR REPLACE PROCEDURE CORE.SP_REFRESH_DOCUMENTS_ENRICHED(P_SIMULATE_ERROR BOOLEAN DEFAULT FALSE)
RETURNS VARIANT
LANGUAGE SQL
AS
$$
DECLARE
    v_started_at    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP();
    v_rows_before   NUMBER        DEFAULT 0;
    v_rows_loaded   NUMBER        DEFAULT 0;
    v_total_amount  NUMBER(38,2)  DEFAULT 0;
    v_result        VARIANT;
    empty_result    EXCEPTION (-20001, 'The view returned no rows, so the table was left as it was');
    simulated_error EXCEPTION (-20002, 'Simulated error after the delete, to show that the rollback restores the table');
BEGIN
    SELECT COUNT(*) INTO :v_rows_before
    FROM FIN_COST_REPORTING.CORE.DOCUMENTS_ENRICHED;

    BEGIN TRANSACTION;

    DELETE FROM FIN_COST_REPORTING.CORE.DOCUMENTS_ENRICHED;

    IF (P_SIMULATE_ERROR) THEN
        RAISE simulated_error;
    END IF;

    INSERT INTO FIN_COST_REPORTING.CORE.DOCUMENTS_ENRICHED
    SELECT v.*, CURRENT_TIMESTAMP()::TIMESTAMP_NTZ
    FROM FIN_COST_REPORTING.CORE.V_DOCUMENTS_ENRICHED v;

    SELECT COUNT(*), COALESCE(SUM(AMOUNT), 0) INTO :v_rows_loaded, :v_total_amount
    FROM FIN_COST_REPORTING.CORE.DOCUMENTS_ENRICHED;

    IF (v_rows_loaded = 0) THEN
        RAISE empty_result;
    END IF;

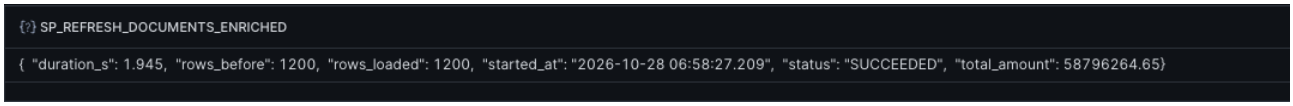
    COMMIT;

      SELECT OBJECT_CONSTRUCT(
          'status',        'SUCCEEDED',
          'rows_before',   :v_rows_before,
          'rows_loaded',   :v_rows_loaded,
          'total_amount',  :v_total_amount,
          'started_at',    :v_started_at,
          'duration_s',    DATEDIFF('millisecond', :v_started_at, CURRENT_TIMESTAMP()) / 1000)
      INTO :v_result;

      RETURN v_result;

  EXCEPTION
      WHEN OTHER THEN
          ROLLBACK;
          RAISE;
  END;
  $$;
```

CALL CORE.SP_REFRESH_DOCUMENTS_ENRICHED();



CALL CORE.SP_REFRESH_DOCUMENTS_ENRICHED(TRUE);



Exercise 4 - Create Snowflake Task

CORE.T_REFRESH_DOCUMENTS_ENRICHED calls the refresh procedure every day at 6:00 UTC+3

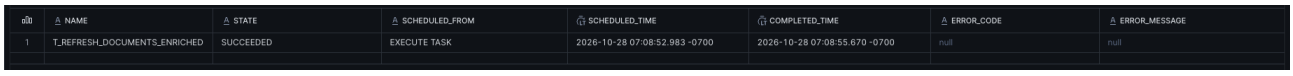
Compute: serverless
The job runs a few seconds, once a day. Serverless compute is billed per second of use. Warehouses are a better choice for long or heavy jobs, or when many tasks run on the same warehouse and keep it busy. Here there is no such load, so serverless is cheaper and needs no warehouse management.

Create task:
```
CREATE OR REPLACE TASK CORE.T_REFRESH_DOCUMENTS_ENRICHED
    USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
    SCHEDULE = 'USING CRON 0 6 * * * EUROPE/BUCHAREST'
    USER_TASK_TIMEOUT_MS = 600000
    COMMENT = 'EXERCISE 4: DAILY FULL REFRESH OF CORE.DOCUMENTS_ENRICHED'
AS
    CALL FIN_COST_REPORTING.CORE.SP_REFRESH_DOCUMENTS_ENRICHED();
```

Manual test:
```
EXECUTE TASK CORE.T_REFRESH_DOCUMENTS_ENRICHED;
```

Activate (by default it's suspended):
```
ALTER TASK CORE.T_REFRESH_DOCUMENTS_ENRICHED RESUME;
```

Check the runs:
```
SELECT NAME, STATE, SCHEDULED_FROM, SCHEDULED_TIME, COMPLETED_TIME, ERROR_CODE, ERROR_MESSAGE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
        TASK_NAME => 'T_REFRESH_DOCUMENTS_ENRICHED',
        RESULT_LIMIT => 20))
ORDER BY SCHEDULED_TIME DESC;
```

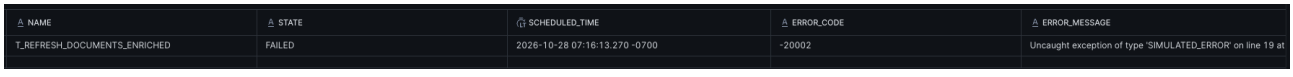


	NAME	STATE	SCHEDULED_FROM	SCHEDULED_TIME	COMPLETED_TIME	ERROR_CODE	ERROR_MESSAGE
1	T_REFRESH_DOCUMENTS_ENRICHED	SUCCEEDED	EXECUTE TASK	2026-10-28 07:08:52.983 -0700	2026-10-28 07:08:55.670 -0700	null	null

Simulate an error:
```
ALTER TASK CORE.T_REFRESH_DOCUMENTS_ENRICHED MODIFY AS
    CALL FIN_COST_REPORTING.CORE.SP_REFRESH_DOCUMENTS_ENRICHED(TRUE);

EXECUTE TASK CORE.T_REFRESH_DOCUMENTS_ENRICHED;

SELECT NAME, STATE, SCHEDULED_TIME, ERROR_CODE, ERROR_MESSAGE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
        TASK_NAME => 'T_REFRESH_DOCUMENTS_ENRICHED',
        ERROR_ONLY => TRUE,
        RESULT_LIMIT => 10))
ORDER BY SCHEDULED_TIME DESC;
```



NAME	STATE	SCHEDULED_TIME	ERROR_CODE	ERROR_MESSAGE
T_REFRESH_DOCUMENTS_ENRICHED	FAILED	2026-10-28 07:16:13.270 -0700	-20002	Uncaught exception of type 'SIMULATED_ERROR' on line 19 at

Exercise 5 - Python in Snowflake

Cost types classification
Classifies every document into a cost type, with rules that finance controllers maintain as data.

New objects:
CORE.RULESETS (table)
- defined rules with condition as JSON:
e.g. {"AccountNumber": "68*", "OrgUnit": "DAI", "ExternalSupplier": false}

CORE.DOCUMENT_COST_TYPES (table)
- stores the procedure results, labelled docs

CORE.SP_CLASSIFY_DOCUMENTS (procedure)
- the python code that:
  - reads the documents and the rules
  - turns each json into a filter
  - applies the filters to the documents and labels the matches
  - saves all matches in CORE.DOCUMENT_COST_TYPES

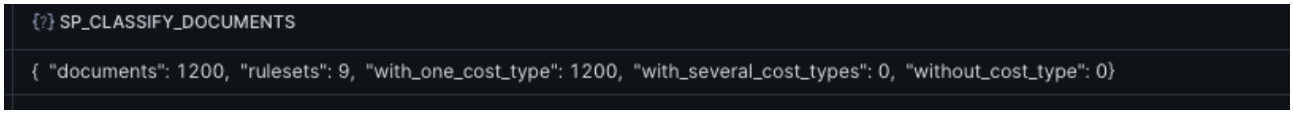
Why python:
The classification rules are stored as JSON and edited by controllers, so each rule can use different fields, lists or wildcards. Python turns each rule into a filter on the documents, with clear error messages for invalid rules. The filters run inside Snowflake, so the data never leaves it. In SQL we would have to assemble the query text by hand, which is harder to read, test and secure.

classify_documents.py
- Reads the documents from CORE.DOCUMENTS_ENRICHED and rulesets from CORE.RULESETS
- Converts each JSON rule into a Snowpark filter
- Applies each filter on the documents and labels the matches with the rule's cost type
- Saves the results in CORE.DOCUMENT_COST_TYPES
- Returns a summary

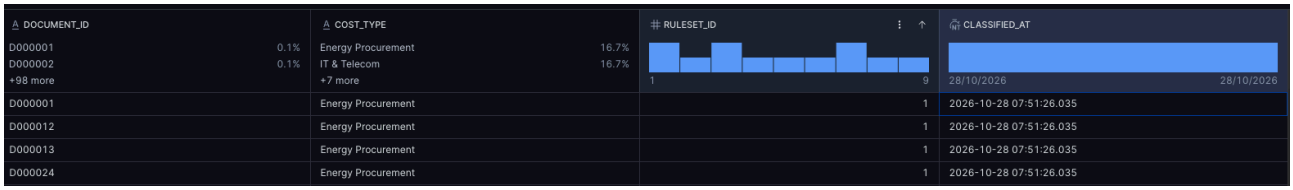
The python file is stored in a stage.

Create procedure:
```
CREATE OR REPLACE PROCEDURE CORE.SP_CLASSIFY_DOCUMENTS()
RETURNS VARIANT
LANGUAGE PYTHON
RUNTIME_VERSION = '3.11'
PACKAGES = ('SNOWFLAKE-SNOWPARK-PYTHON')
IMPORTS = ('@FIN_COST_REPORTING.CORE.PYTHON_CODE/CLASSIFY_DOCUMENTS.PY')
HANDLER = 'CLASSIFY_DOCUMENTS.CLASSIFY';


CALL CORE.SP_CLASSIFY_DOCUMENTS();
```



Results:



Exercise 6 - Streamlit

Cost overview filters, KPIs, cost per period and cost type, documents table, drill-down, export Rules, upload changed rules, re-classify.

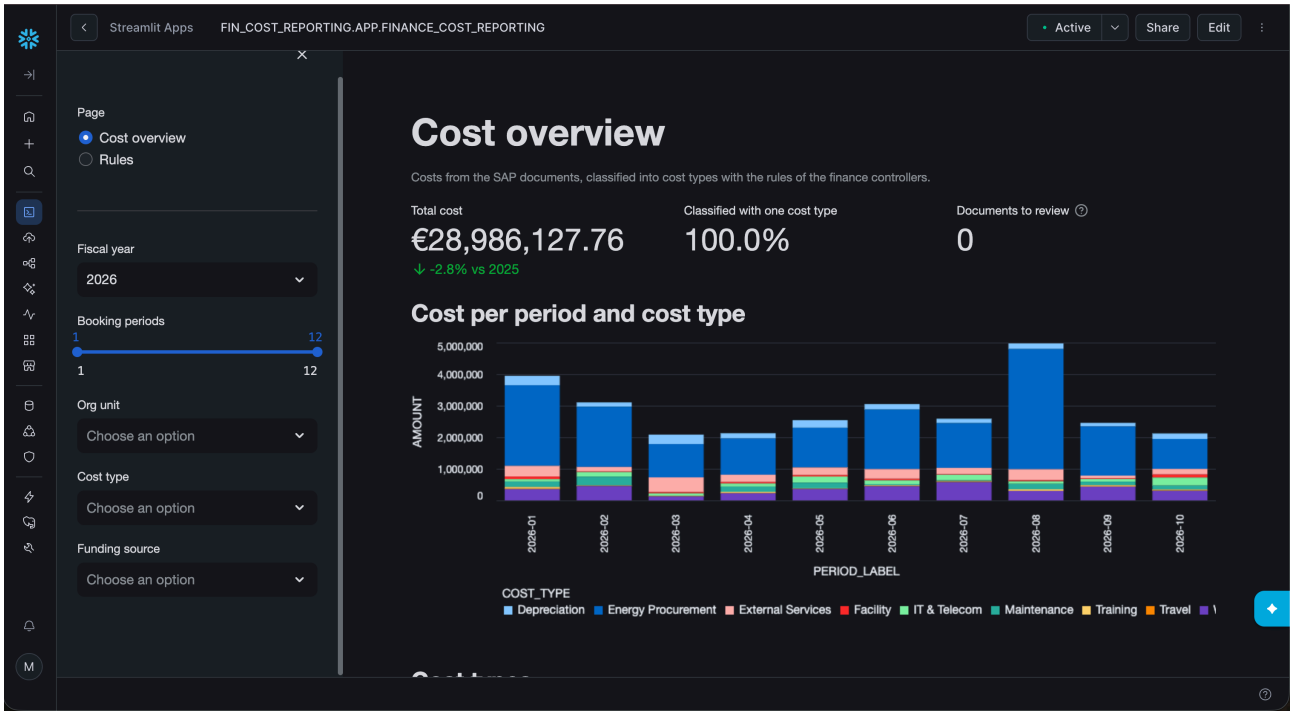
The app contains 2 pages:
- Cost overview:
  - filters (year, period, org unit, cost type, funding source)
  - KPIs
  - Cost per period and cost type
  - Documents table
  - CSV export
  - Why the document got its cost type
- Rules
  - Download the rules as excel
  - Upload changed rules
  - Classify again (trigger the procedure from exercise 5)

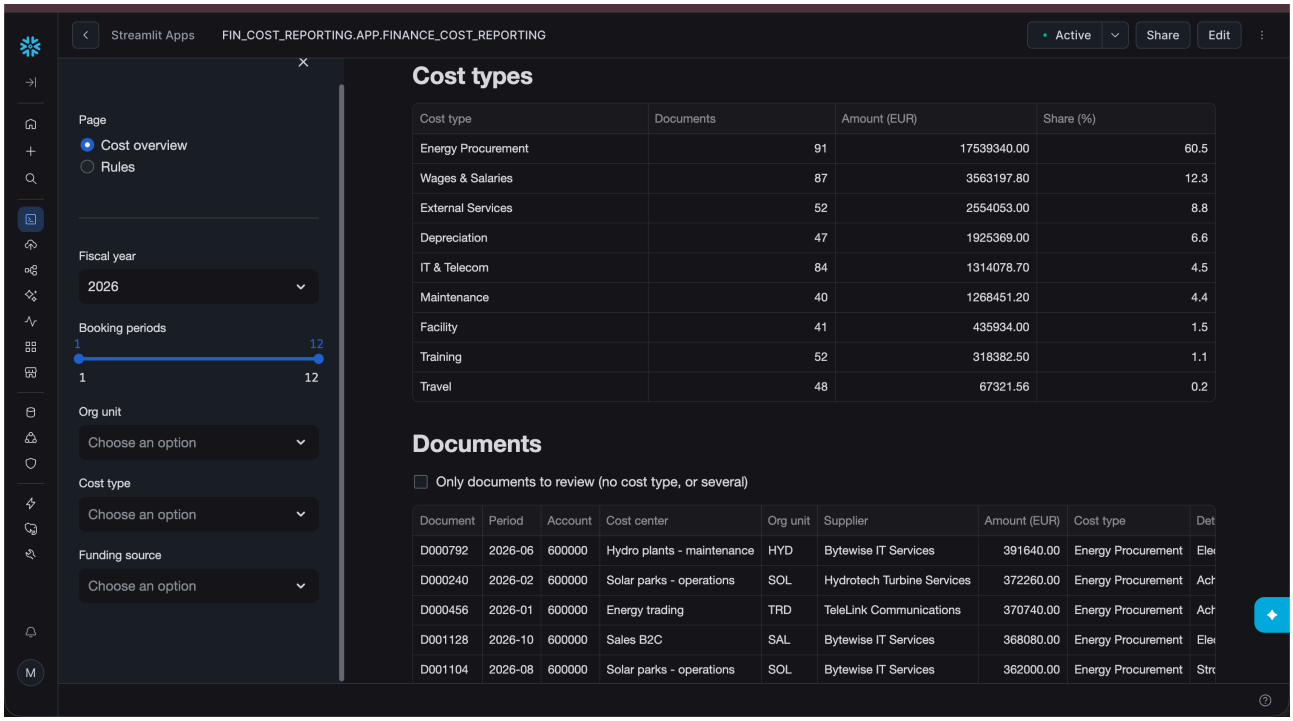
The app is deployed in snowflake in a stage and can be accessed from Snowsight.

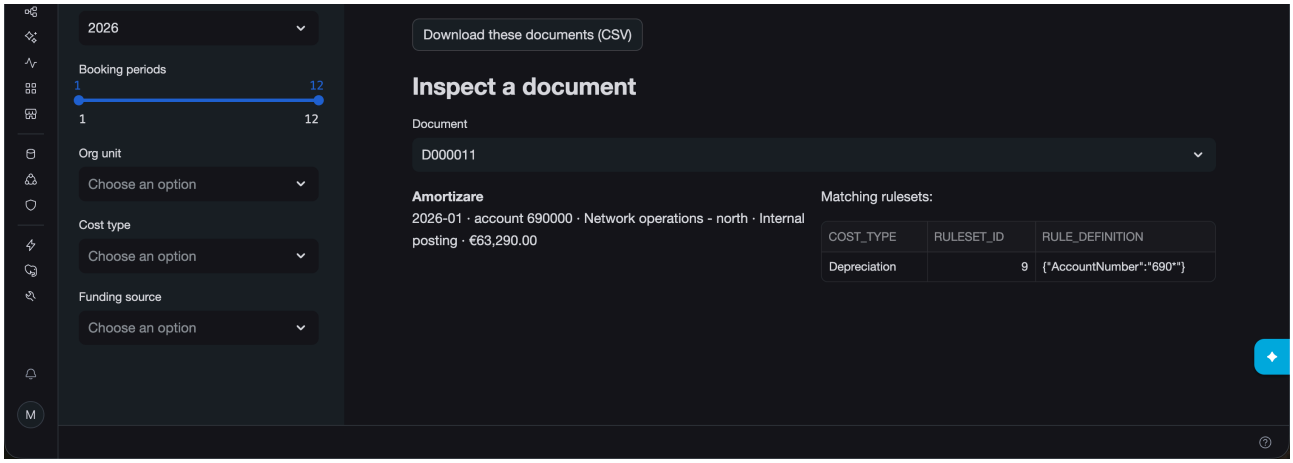
```
CREATE STAGE IF NOT EXISTS APP.STREAMLIT_STAGE
    DIRECTORY = (ENABLE = TRUE)
    ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE');

CREATE OR REPLACE STREAMLIT APP.FINANCE_COST_REPORTING
    FROM '@FIN_COST_REPORTING.APP.STREAMLIT_STAGE/COST_APP'
    MAIN_FILE = 'STREAMLIT_APP.PY'
    QUERY_WAREHOUSE = COMPUTE_WH;
```

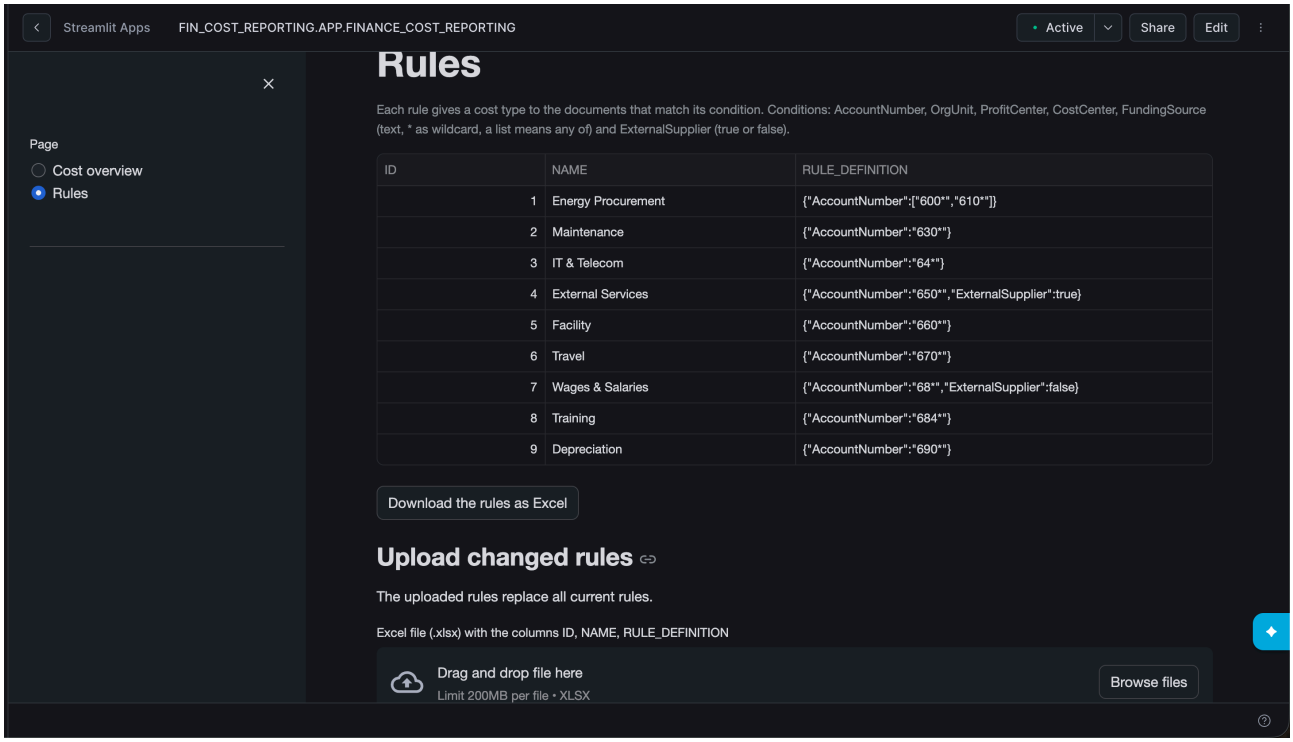
Cost Overview Page:




Streamlit Apps
FIN_COST_REPORTING.APP.FINANCE_COST_REPORTING
Active
Share
Edit
Page
Cost overview
Rules
Fiscal year
2026
Booking periods
1
12
1
12
Org unit
Choose an option
Cost type
Choose an option
Funding source
Choose an option
Cost types
Cost type | Documents | Amount (EUR) | Share (%)
Energy Procurement | 91 | 17539340.00 | 60.5
Wages & Salaries | 87 | 3563197.80 | 12.3
External Services | 52 | 2554053.00 | 8.8
Depreciation | 47 | 1925369.00 | 6.6
IT & Telecom | 84 | 1314078.70 | 4.5
Maintenance | 40 | 1268451.20 | 4.4
Facility | 41 | 435934.00 | 1.5
Training | 52 | 318382.50 | 1.1
Travel | 48 | 67321.56 | 0.2
Documents
Only documents to review (no cost type, or several)
Document | Period | Account | Cost center | Org unit | Supplier | Amount (EUR) | Cost type
D000792 | 2026-06 | 600000 | Hydro plants - maintenance | HYD | Bytewise IT Services | 391640.00 | Energy Procurement
D000240 | 2026-02 | 600000 | Solar parks - operations | SOL | Hydrotech Turbine Services | 372260.00 | Energy Procurement
D000456 | 2026-01 | 600000 | Energy trading | TRD | TeleLink Communications | 370740.00 | Energy Procurement
D001128 | 2026-10 | 600000 | Sales B2C | SAL | Bytewise IT Services | 368080.00 | Energy Procurement
D001104 | 2026-08 | 600000 | Solar parks - operations | SOL | Bytewise IT Services | 362000.00 | Energy Procurement


2026
Booking periods
1
12
1
12
Org unit
Choose an option
Cost type
Choose an option
Funding source
Choose an option
Download these documents (CSV)
Inspect a document
Document
D000011
Amortizare
2026-01 · account 690000 · Network operations - north · Internal posting · €63,290.00
Matching rulesets:
COST_TYPE | RULESET_ID | RULE_DEFINITION
Depreciation | 9 | {"AccountNumber":"690*"}

Rules page:


Streamlit Apps
FIN_COST_REPORTING.APP.FINANCE_COST_REPORTING
Active
Share
Edit
Page
Cost overview
Rules
Rules
Each rule gives a cost type to the documents that match its condition. Conditions: AccountNumber, OrgUnit, ProfitCenter, CostCenter, FundingSource (text, * as wildcard, a list means any of) and ExternalSupplier (true or false).
ID | NAME | RULE_DEFINITION
1 | Energy Procurement | {"AccountNumber":["600*","610*"]}
2 | Maintenance | {"AccountNumber":"630*"}
3 | IT & Telecom | {"AccountNumber":"64*"}
4 | External Services | {"AccountNumber":"650*","ExternalSupplier":true}
5 | Facility | {"AccountNumber":"660*"}
6 | Travel | {"AccountNumber":"670*"}
7 | Wages & Salaries | {"AccountNumber":"68*","ExternalSupplier":false}
8 | Training | {"AccountNumber":"684*"}
9 | Depreciation | {"AccountNumber":"690*"}
Download the rules as Excel
Upload changed rules
The uploaded rules replace all current rules.
Excel file (.xlsx) with the columns ID, NAME, RULE_DEFINITION
Drag and drop file here
Limit 200MB per file • XLSX
Browse files